

DIP Free Program Overview

Deliberative Intelligence Protocol - Local Decision Analysis Tool

Generated for the local DIP free edition. The document describes the full flow from query intake to report rendering.

Project Summary

DIP Arena is a local, offline decision intelligence system that routes a question through multiple specialist agents, critiques high-confidence claims, validates evidence against a small built-in knowledge base, and synthesizes a structured report for review.

System Flow

Input	Query, domain, stakes, constraints, notes
Route	Orchestrator selects research, risk, ethics, policy, economics, and strategy agents
Deliberate	Agents respond in parallel with claims, evidence, assumptions, risks, and confidence
Challenge	Critiques question blind spots and unsupported certainty
Validate	Each claim is scored against the retrieved evidence
Synthesize	The final DecisionReport aggregates recommendation, confidence, claims, dissent, and risks
Present	CLI prints JSON and the web UI renders the report with history and comparison tools

Component Overview

Layer	Primary Function
CLI	Accepts a query, runs the orchestrator, prints JSON, and optionally saves a report.
Web Server	Serves the SPA and exposes /api/analyze for browser-driven analysis.
Orchestrator	Coordinates observe, retrieve, run, challenge, validate, and synthesize steps.
Agents	Provide specialized perspectives for research, risk, ethics, policy, economics, and strategy.
Models	Define the typed data structures that move through the pipeline.
Frontend	Collects input, renders reports, preserves history, and exports artifacts.

Component and Function Reference

This section lists the role of each source component and the function-level behavior it contributes to the product.

dip_workflow.py

Thin convenience entrypoint for command-line launches.

- **main()**: Imports and delegates to dip_free.cli.main so ``python dip_workflow.py ...`` works without extra wiring.

dip_free/__main__.py

Package entrypoint for `python -m dip_free`.

- **main()**: Forwards execution to the CLI layer when the package is launched as a module.

dip_free/cli.py

Command-line interface and JSON report export.

- **report_to_dict(report)**: Serializes a DecisionReport dataclass into a JSON-friendly dictionary.
- **save_report(report_data, output_path)**: Writes report JSON to a requested path or a timestamped file in reports/.
- **main()**: Parses CLI arguments, runs the orchestrator, prints JSON, and optionally saves the report.

dip_free/orchestrator.py

Decision workflow controller that routes, critiques, validates, and synthesizes results.

- **DIPOrchestrator.__init__**: Builds the default knowledge base and prepares the specialist agent pool.
- **decide(query, domain, context)**: Runs the full observe/retrieve/agent/challenge/validate/synthesize pipeline.
- **_observe(query, domain, context)**: Normalizes input into a Task object and chooses required agents.
- **_retrieve_context(task)**: Collects domain guidance, notes, and task metadata for downstream reasoning.
- **_route_agents(domain, query)**: Selects the specialist agents that should participate in the analysis.
- **_run_agents(task, context)**: Runs the chosen agents in parallel with a thread pool and sorts their responses.
- **_challenge(task, agent_outputs)**: Creates critique objects that question overconfidence and open gaps.
- **_validate(task, agent_outputs, context)**: Scores each claim against the retrieved evidence.
- **_score_claim(claim, source_pool, agent_confidence)**: Assigns supported, weakly_supported, or unsupported status and a confidence adjustment.
- **_synthesize(task, agent_outputs, critiques, validations)**: Merges all signals into the final DecisionReport.
- **_recommendation_from_domain(domain, agent_outputs)**: Chooses a domain-specific recommendation or the strongest stance.
- **_merge_unique(values)**: Deduplicates lists while preserving order.

These are the highest-impact control points in the application and drive most of the user-visible behavior.

dip_free/agents.py

Specialist reasoning agents with focused perspectives.

- **BaseAgent.__init__**: Stores the shared knowledge base for all agents.
- **BaseAgent.run**: Abstract interface that each concrete agent overrides.
- **BaseAgent._evidence_for_domain(task, fallback)**: Returns domain evidence or a safe fallback string.
- **ResearchAgent.run**: Focuses on evidence-first reasoning and missing-fact discovery.
- **RiskAgent.run**: Highlights failure modes, uncertainty, and downside exposure.
- **EthicsAgent.run**: Checks fairness, harm minimization, and stakeholder impact.
- **PolicyAgent.run**: Looks at governance, compliance, auditability, and rollout structure.
- **EconomicsAgent.run**: Weighs cost, benefit, externalities, and long-run value.
- **StrategyAgent.run**: Checks alignment with objectives, execution capacity, and trust.

dip_free/models.py

Shared data model layer for tasks, agent responses, critiques, validations, and reports.

- **Domain:** Typed literal for supported domains: general, policy, healthcare, business, research, security.
- **AgentKind:** Typed literal for the six specialist agent categories.
- **ValidationStatus:** Typed literal for claim status values: supported, weakly_supported, unsupported.
- **Task:** Captures the query, stakes, constraints, routing, evidence needs, and context.
- **AgentResponse:** Stores each agent's stance, claims, evidence, assumptions, confidence, risks, and open questions.
- **Critique:** Represents a challenge to an agent's output, including severity and unresolved issue.
- **ValidationResult:** Records how a claim was scored and why.
- **DecisionReport:** Holds the final recommendation, confidence, supporting claims, dissent, risks, tradeoffs, and diagnostics.
- **ReportDict:** Dictionary alias used for serialized report data.

dip_web_server.py

Local HTTP server that serves the UI and analysis API.

- **DIPWebHandler.do_GET:** Serves health, HTML, CSS, and JS responses.
- **DIPWebHandler.do_POST:** Accepts analysis requests, normalizes payloads, and returns a report JSON response.
- **DIPWebHandler._serve_file:** Reads a static file and returns it with the correct content type.
- **DIPWebHandler._send_json:** Serializes Python data to JSON and writes the HTTP response.
- **DIPWebHandler.log_message:** Suppresses default HTTP server logging for a cleaner console.
- **main():** Parses host/port arguments and starts the ThreadingHTTPServer.

These are the highest-impact control points in the application and drive most of the user-visible behavior.

web/app.js

Browser-side state, rendering, export, history, comparison, and theme behavior.

- **setStatus(message, tone):** Updates the status banner and stores a tone token.
- **getTheme():** Reads the saved theme from localStorage.
- **setTheme(theme):** Applies the theme to the document and updates the toggle label.
- **toggleTheme():** Switches between the two saved themes.
- **clearElement(element, fallbackText):** Clears a container and optionally marks it as empty state.
- **createItem(text):** Builds a simple list item for summaries and results.
- **createChip(text):** Builds a pill-shaped chip for tradeoffs and triggers.
- **toMarkdown(report):** Exports the active report to a Markdown document.
- **escapeHtml(value):** Escapes user-facing text before HTML insertion.
- **getHistory():** Loads saved reports from localStorage.
- **saveHistory(history):** Persists the latest report history list locally.
- **addToHistory(entry):** Adds the newest analysis to history and refreshes comparison options.

- **renderHistory()**: Renders saved queries as clickable history items.
- **renderCompareOptions()**: Populates the Report A and Report B selectors.
- **populateFormFromInput(input)**: Restores a saved query into the form controls.
- **downloadReport(report)**: Downloads the JSON report as a file.
- **downloadMarkdown(report)**: Downloads the report as Markdown.
- **renderList(element, items, emptyText)**: Renders a simple list or fallback empty state.
- **renderChips(element, items, emptyText)**: Renders a chip group or fallback empty state.
- **renderAgents(items)**: Builds the agent debate cards.
- **findHistoryItem(id)**: Finds a saved report by ID.
- **renderComparison(left, right)**: Builds the side-by-side comparison view.
- **renderValidations(items)**: Builds validation cards with status badges.
- **renderReport(report, enableExport)**: Populates the dashboard with the latest analysis result.
- **analyzeQuery(payload)**: Sends the request to /api/analyze and handles success or failure.

These are the highest-impact control points in the application and drive most of the user-visible behavior.

[web/index.html](#)

Semantic page structure for the navbar, hero, forms, report area, and support sections.

- **No executable functions**: Provides anchors, layout containers, and semantic sections used by app.js and styles.css.

[web/styles.css](#)

Design system, palette, responsive layout, and component styling.

- **No executable functions**: Defines the visual behavior of the premium SaaS interface, including layout, cards, spacing, and theme tokens.

[sample_inputs/delhi_policy.json](#)

Ready-to-use example payload for testing the policy flow.

- **No functions**: Static input data for a representative policy scenario.

[evaluation/benchmark_queries.json](#)

Benchmark query set for experimentation and regression checks.

- **No functions**: Static benchmark data for quality checks and comparisons.

[reports/](#)

Local output directory for saved decision reports.

- **No functions**: Stores JSON exports generated from CLI or the browser app.

Agent and Report Outputs

The report format is intentionally explicit so the user can inspect what was considered, what was challenged, and what would change the decision.

- **recommendation** - The selected path or stance for the query.
- **confidence_score** - Weighted signal combining agent confidence, evidence coverage, and critique penalties.
- **supporting_claims** - Claims that passed validation most strongly.
- **dissenting_views** - Views that were uncertain, risky, or divergent.
- **risks** - Consolidated risks from the specialist outputs and critiques.
- **tradeoffs** - Explicit tension points such as speed versus scrutiny.
- **what_would_change_the_decision** - Conditions that would materially alter the recommendation.
- **agent_outputs** - The full per-agent debate output.
- **critiques** - Challenges raised against overconfident assumptions.
- **validations** - Claim-by-claim evidence scoring.

Runtime Commands

CLI

```
python -m dip_free "Should Delhi prioritize economic growth or pollution control?" --domain policy --stakes high --constraint "must remain auditable" --free --save-report
```

Web

```
python dip_web_server.py --port 8000  
Open http://127.0.0.1:8000 in a browser
```

Browser UI Features

- Query entry with local analysis
- JSON and Markdown export
- Local browser history with restore support
- Side-by-side report comparison
- Theme toggle and premium dashboard styling
- Responsive layout for desktop and mobile

Design Notes

The free edition keeps the interface offline, auditable, and deterministic. The browser front end presents the same underlying workflow as a polished SaaS dashboard while the backend stays deliberately small and transparent.

Generated on June 13, 2026 at 01:41